

Name – Manjushree

PetStore CRUD API Automation Report

Githublink -

Project Overview

This project automates CRUD (Create, Read, Update, Delete) operations on the Swagger PetStore API using Rest Assured and JUnit 5.

Base URL

<https://petstore.swagger.io/v2>

Tools & Technologies

- Java
 - Rest Assured
 - JUnit 5
 - Maven
 - Swagger PetStore API
-

Test Scenarios Implemented

1. Create Pet

Endpoint

POST /pet

Request Body

```
{  
  "id": 500,  
  "name": "Chinnu",  
  "status": "available"  
}
```

Validation

- Verify status code is 200
- Verify pet name is Chinnu

- Verify status is available

Result

PASS

Code	Details
200 <i>Undocumented</i>	Response body <pre>{ "id": 500, "category": { "id": 500, "name": "Chinnu" }, "name": "doggie", "photoUrls": ["string"], "tags": [{ "id": 500, "name": "Chinnu" }], "status": "available" }</pre>

2. Update Pet

Endpoint

PUT /pet

Request Body

```
{
  "id": 500,
  "name": "Bruno",
  "status": "available"
}
```

Validation

- Verify status code is 200
- Verify pet name is Bruno
- Verify status is available

Result

PASS

Code	Details
200 <i>Undocumented</i>	Response body <pre>{ "id": 500, "category": { "id": 500, "name": "Chinnu" }, "name": "doggie", "photoUrls": ["string"], "tags": [{ "id": 500, "name": "Chinnu" }], "status": "available" }</pre>

3. Update Pet Status to Sold

Endpoint

PUT /pet

Request Body

```
{
  "id": 500,
  "name": "Bruno",
  "status": "sold"
}
```

Validation

- Verify status code is 200
- Verify status is sold

Result

PASS

```
    status : sold
  },
  {
    "id": 500,
    "category": {
      "id": 500,
      "name": "Bruno"
    },
    "name": "doggie",
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 500,
        "name": "Bruno"
      }
    ],
    "status": "sold"
  },
  ]
}
```

4. Find Pet By Status

Endpoint

GET /pet/findByStatus?status=sold

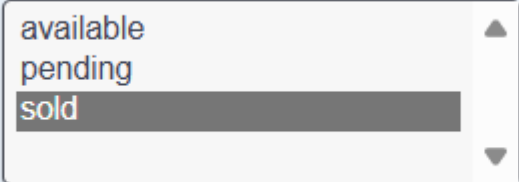
Validation

- Verify status code is 200
- Verify sold pets are returned

Result

PASS

Parameters

Name	Description
status * required array<string> (query)	Status values that need to be considered for filter 

```
    status : sold
  },
  {
    "id": 500,
    "category": {
      "id": 500,
      "name": "Bruno"
    },
    "name": "doggie",
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 500,
        "name": "Bruno"
      }
    ],
    "status": "sold"
  },
]
```

5. Delete Pet

Endpoint

DELETE /pet/500

Validation

- Verify status code is 200

Result

PASS

Framework Structure

src

└─ test

└─ java

└─ apiCrudTest

└─ test.java

pom.xml

Test Data

Field	Value
Pet ID	500
Initial Name	Chinnu
Updated Name	Bruno
Initial Status	available
Final Status	sold

Assertions Used

```
.statusCode(200)
.body("name", equalTo("Chinnu"))
.body("status", equalTo("available"))
.statusCode(200)
.body("name", equalTo("Bruno"))
.body("status", equalTo("sold"))
```

Execution Steps

1. Configure Maven dependencies.

2. Set Base URI and Base Path.
 3. Execute CRUD test methods.
 4. Validate API responses using assertions.
 5. Review execution results.
-

Conclusion

The PetStore API CRUD automation framework was successfully implemented using Rest Assured and JUnit 5. The framework validates Create, Update, Read, and Delete operations and verifies API responses using assertions. All test scenarios executed successfully with expected results.